Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Laboratoriya İşi № 1

Tələbənin adı və soyadı: Quliyeva Fatimə

Qrup: KOM25

Müəllim: Şamil Hümbətov

Tarix: 12 oktyabr 2026

2026

Tapşırıq 1: Aptekdə növbə

1. Başlıq: “Bağlı siyahı əsasında növbə (Queue) strukturunun və FIFO prinsipinin aptek növbəsi modelində reallaşdırılması”

2. Məqsədlər:

- Növbə strukturunu və FIFO prinsipini başa düşmək
- Bağlı siyahı ilə növbə qurmaq (bas/son göstəriciləri)
- Obyekt yönümlü proqramlaşdırma elementlərini (sinif, konstruktor) tətbiq etmək
- Real həyat məsələsini verilənlər strukturu ilə modelləşdirmək

3. Problemin təhlili:

- **Giriş:** müştərinin adı və dərman adı. **Çıxış:** növbənin vəziyyəti, xidmət olunan müştəri haqqında mesaj.
- **Məhdudiyyətlər:** əlavə yalnız sondan, çıxarma yalnız baştan. Boş növbədən çıxarmaq xəta verməməlidir.
- **Seçilmiş struktur:** bas və son göstəricili bağlı siyahı. Səbəb: list.pop(0) **O(n)`-dir, bağlı siyahıda isə əlavə və çıxarma **O(1)**.

Əməliyyat Mürəkkəblik

elave_et	O(1)
xidmet_et	O(1)
goster	O(n)

4. Psevdokod:

```
SİNİF Musteri: ad, derman

SİNİF Node:    musteri, next ← NULL

SİNİF Novbe:   bas ← NULL, son ← NULL, say ← 0


FUNKSİYA elave_et(ad, derman)

   yeni_node ← Node(Musteri(ad, derman))

   ƏGƏR bas = NULL ONDA

      bas ← yeni_node

      son ← yeni_node
```

```
    ƏKS HALDA
        son.next ← yeni_node
        son ← yeni_node
    SON ƏGƏR
    say ← say + 1
SON FUNKSİYA

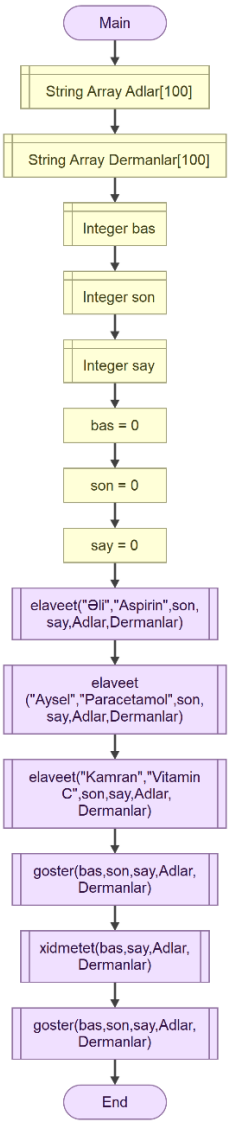
FUNKSİYA xidmet_et()
    ƏGƏR bas = NULL ONDA
        ÇAP ET "Növbə boşdur"
        QAYTAR
    SON ƏGƏR
    cixan ← bas
    bas ← bas.next
    ƏGƏR bas = NULL ONDA
        son ← NULL
    SON ƏGƏR
    say ← say − 1
    ÇAP ET cixan.musteri.ad, cixan.musteri.derman
SON FUNKSİYA

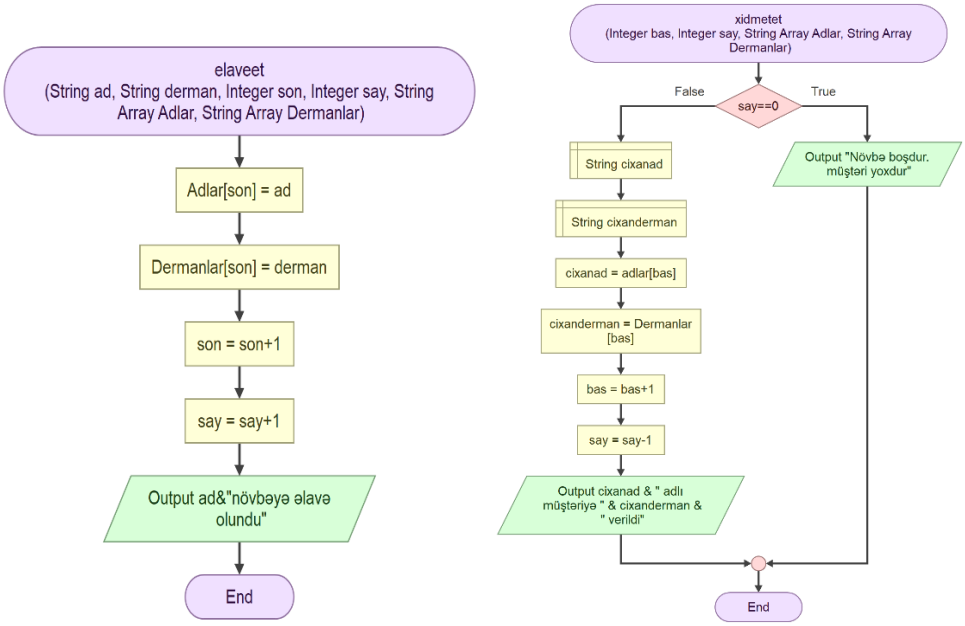
FUNKSİYA goster()
    ƏGƏR bas = NULL ONDA
        ÇAP ET "Növbə boşdur"
        QAYTAR
    SON ƏGƏR
    cari ← bas
    sira ← 1
    ƏVVƏLCƏ cari ≠ NULL DÖVR
        ÇAP ET sira, cari.musteri.ad, cari.musteri.derman
        cari ← cari.next
        sira ← sira + 1
```

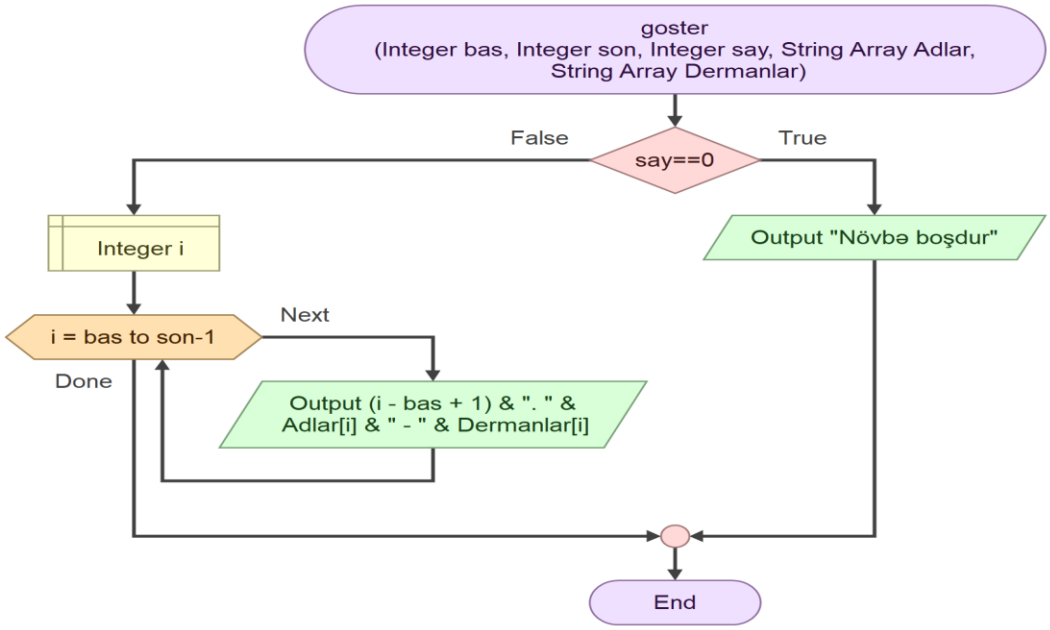
SON DÖVR

SON FUNKSİYA

5. Blok-sxem:







6. Proqramlaşdırma:

```
class Musteri:
    def __init__(self, ad, derman):
        self.ad = ad
        self.derman = derman

class Node:
    def __init__(self, musteri):
```

```
        self.musteri = musteri
        self.next = None

class Novbe:
    def __init__(self):
        self.bas = None
        self.son = None
        self.say = 0

    def elave_et(self, ad, derman):
        yeni_musteri = Musteri(ad, derman)
        yeni_node = Node(yeni_musteri)
        if self.bas is None:
            self.bas = self.son = yeni_node
        else:
            self.son.next = yeni_node
            self.son = yeni_node
        self.say += 1
        print(f"{ad} növbəyə əlavə olundu")

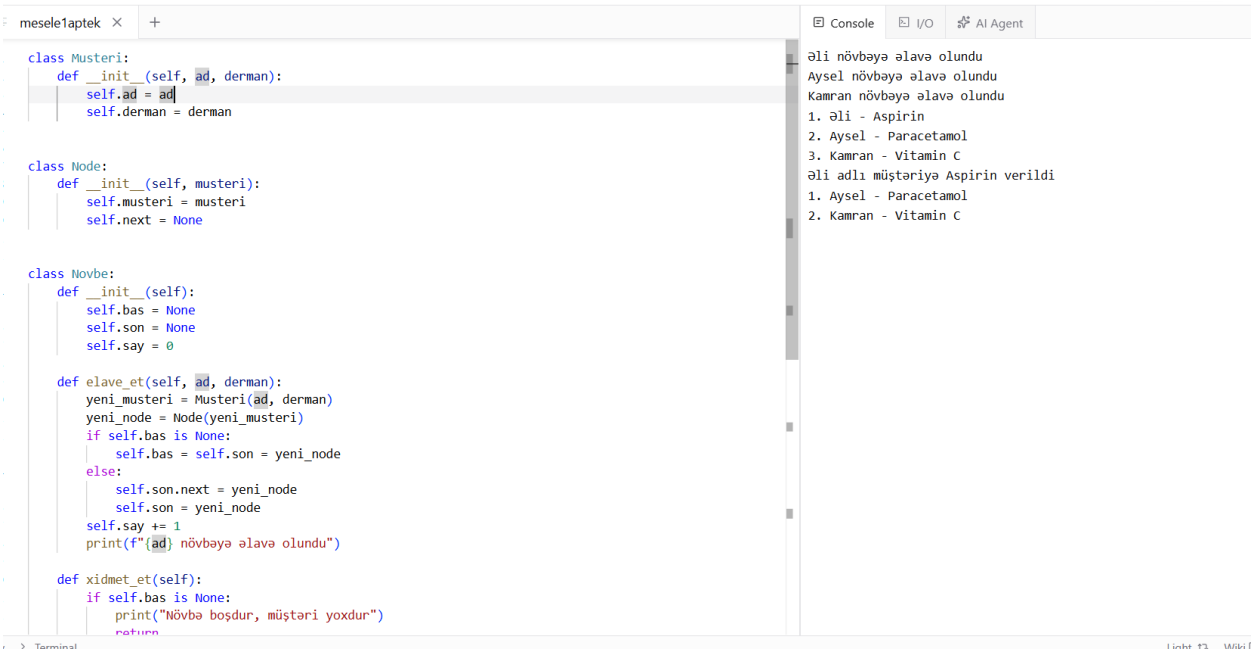
    def xidmet_et(self):
        if self.bas is None:
            print("Növbə boşdur, müştəri yoxdur")
            return
        cixan = self.bas
        self.bas = self.bas.next
        if self.bas is None:
            self.son = None
        self.say -= 1
        musteri = cixan.musteri
        print(f"{musteri.ad} adlı müştəriyə {musteri.derman} verildi")

    def goster(self):
        if self.bas is None:
            print("Növbə boşdur")
```

```
        return
    cari = self.bas
    sira = 1
    while cari is not None:
        print(f"{sira}. {cari.musteri.ad} - {cari.musteri.derman}")
        cari = cari.next
        sira += 1


if __name__ == "__main__":
    novbe = Novbe()
    novbe.elave_et("Əli", "Aspirin")
    novbe.elave_et("Aysel", "Paracetamol")
    novbe.elave_et("Kamran", "Vitamin C")
    novbe.goster()
    novbe.xidmet_et()
    novbe.goster()
```

7. Test nəticələri:



Tapşırıq 2: Avtobus dayanacağı

1. Başlıq: “İki növbə əsasında prioritetli xidmət sisteminin avtobus dayanacağı modelində reallaşdırılması”

2. Məqsədlər: prioritetli növbə məntiqini anlamaq; bir neçə növbəni birlikdə idarə etmək; kod təkrarından qaçıb sinfi yenidən istifadə etmək (Novbe iki dəfə istifadə olunur); məsuliyyətlərin bölünməsi (Novbe = növbə, Dayanacaq = prioritet).

3. Problemin təhlili:

- **Giriş:** sərnişinin adı və yaşı. **Çıxış:** minən sərnişin, iki növbənin siyahısı.
- **Məhdudiyyətlər:** yaşı 65-dən yuxarı olanlar yaşlılar növbəsinə düşür (tapşırıqda “yuxarı” yazılıb, >); hər növbədə FIFO qorunur; yaş yoxlanır (mənfi, 120-dən böyük və ədəd olmayan dəyərlər rədd edilir).
- **Struktur:** iki Novbe obyekti + Dayanacaq sinfi. Hər əməliyyat O(1), goster O(n).

4. Psevdokod:

```
SABİT YAS_HEDDI ← 65
yaslilar_novbesi ← Novbe()
normal_novbe ← Novbe()

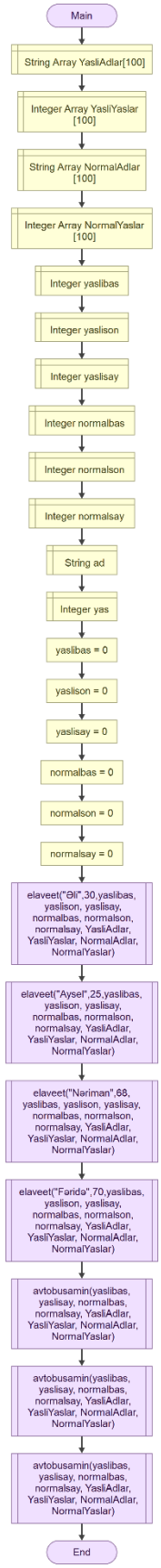
FUNKSİYA elave_et(ad, yas)
    ƏGƏR yas ədəd deyil VƏ YA yas < 0 VƏ YA yas > 120 ONDA
        ÇAP ET "Yaş yanlışdır"
        QAYTAR
    SON ƏGƏR
    sernisin ← Sernisin(ad, yas)
    ƏGƏR yas > YAS_HEDDI ONDA
        yaslilar_novbesi.elave_et(sernisin)
    ƏKS HALDA
        normal_novbe.elave_et(sernisin)
    SON ƏGƏR
SON FUNKSİYA

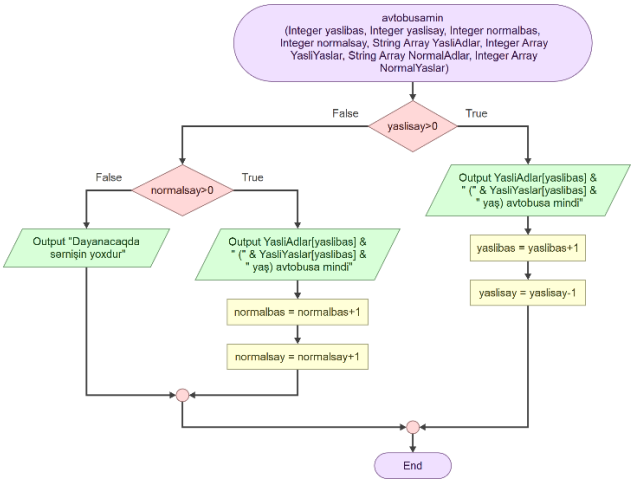
FUNKSİYA avtobusa_min()
    sernisin ← yaslilar_novbesi.cixar()
    ƏGƏR sernisin = NULL ONDA
```

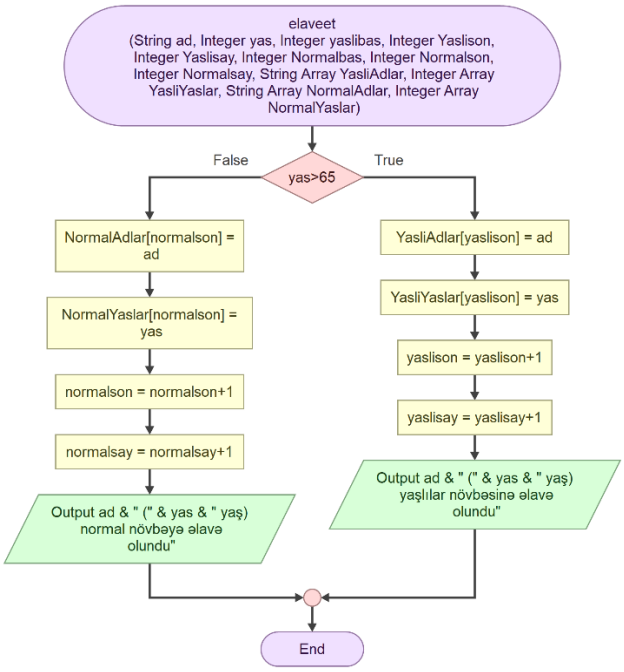
```
        sernisin ← normal_novbe.cixar()
    SON ƏGƏR
    ƏGƏR sernisin = NULL ONDA
        ÇAP ET "Sərnişin yoxdur"
        QAYTAR
    SON ƏGƏR
    ÇAP ET sernisin.ad, "avtobusa mindi"
SON FUNKSİYA

FUNKSİYA goster()
    yaslilar_novbesi.goster()
    normal_novbe.goster()
SON FUNKSİYA
```

5. Blok-sxem:


Main
String Array YasliAdlar[100]
Integer Array YasliYaslar [100]
String Array NormalAdlar [100]
Integer Array NormalYaslar [100]
Integer yaslibas
Integer yaslison
Integer yaslisay
Integer normalbas
Integer normalson
Integer normalsay
String ad
Integer yas
yaslibas = 0
yaslison = 0
yaslisay = 0
normalbas = 0
normalson = 0
normalsay = 0
elaveet("Əli",30,yaslibas, yaslison, yaslisay, normalbas, normalson, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
elaveet("Aysel",25,yaslibas, yaslison, yaslisay, normalbas, normalson, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
elaveet("Nəriman",68, yaslibas, yaslison, yaslisay, normalbas, normalson, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
elaveet("Fərida",70,yaslibas, yaslison, yaslisay, normalbas, normalson, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
avtobusamin(yaslibas, yaslisay, normalbas, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
avtobusamin(yaslibas, yaslisay, normalbas, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
avtobusamin(yaslibas, yaslisay, normalbas, normalsay, YasliAdlar, YasliYaslar, NormalAdlar, NormalYaslar)
End


avtobusamin (Integer yaslibas, Integer yaslisay, Integer normalbas, Integer normalsay, String Array YasliAdlar, Integer Array YasliYaslar, String Array NormalAdlar, Integer Array NormalYaslar)
yaslisay>0
False
True
normalsay>0
False
True
Output "Dayanacaqda sərnişin yoxdur"
Output YasliAdlar[yaslibas] & " (" & YasliYaslar[yaslibas] & " yaş) avtobusa mindi"
normalbas = normalbas+1
normalsay = normalsay+1
Output YasliAdlar[yaslibas] & " (" & YasliYaslar[yaslibas] & " yaş) avtobusa mindi"
yaslibas = yaslibas+1
yaslisay = yaslisay-1
End



6. Proqramlaşdırma:

```
class Sernisin:
    def __init__(self, ad, yas):
        self.ad = ad
        self.yas = yas


class Node:
    def __init__(self, sernisin):
        self.sernisin = sernisin
        self.next = None


class Novbe:
    def __init__(self):
        self.bas = None
        self.son = None
        self.say = 0


    def bosdur(self):
        return self.bas is None
```

```
    def elave_et(self, sernisin):
        yeni_node = Node(sernisin)
        if self.bas is None:
            self.bas = self.son = yeni_node
        else:
            self.son.next = yeni_node
            self.son = yeni_node
        self.say += 1

    def cixar(self):
        if self.bas is None:
            return None
        cixan = self.bas
        self.bas = self.bas.next
        if self.bas is None:
            self.son = None
        self.say -= 1
        return cixan.sernisin

    def goster(self):
        if self.bas is None:
            print("  (boşdur)")
            return
        cari = self.bas
        sira = 1
        while cari is not None:
            print(f"  {sira}. {cari.sernisin.ad} ({cari.sernisin.yas} yaş)")
            cari = cari.next
            sira += 1

class Dayanacaq:
    YAS_HEDDI = 65

    def __init__(self):
        self.yaslilar_novbesi = Novbe()
```

```
        self.normal_novbe = Novbe()

    def elave_et(self, ad, yas):
        if not isinstance(yas, int) or yas < 0 or yas > 120:
            print("Daxil edilən yaş yanlışdır!")
            return
        sernisin = Sernisin(ad, yas)
        if yas > Dayanacaq.YAS_HEDDI:
            self.yaslilar_novbesi.elave_et(sernisin)
            print(f"{ad} ({yas} yaş) yaşlılar növbəsinə əlavə olundu")
        else:
            self.normal_novbe.elave_et(sernisin)
            print(f"{ad} ({yas} yaş) normal növbəyə əlavə olundu")

    def avtobusa_min(self):
        sernisin = self.yaslilar_novbesi.cixar()
        if sernisin is None:
            sernisin = self.normal_novbe.cixar()
        if sernisin is None:
            print("Dayanacaqda sərnişin yoxdur")
            return
        print(f"{sernisin.ad} ({sernisin.yas} yaş) avtobusa mindi")

    def goster(self):
        print("Yaşlılar növbəsi:")
        self.yaslilar_novbesi.goster()
        print("Normal növbə:")
        self.normal_novbe.goster()

if __name__ == "__main__":
    dayanacaq = Dayanacaq()
    dayanacaq.elave_et("Əli", 30)
    dayanacaq.elave_et("Aysel", 25)
    dayanacaq.elave_et("Nəriman", 68)
    dayanacaq.elave_et("Fəridə", 70)
```

```
dayanacaq.goster()

dayanacaq.avtobusa_min()

dayanacaq.avtobusa_min()

dayanacaq.avtobusa_min()

dayanacaq.goster()
```

7. Test nəticələri:



mesele2sernisin

```
class Sernisin:
    def __init__(self, ad, yas):
        self.ad = ad
        self.yas = yas


class Node:
    def __init__(self, sernisin):
        self.sernisin = sernisin
        self.next = None


class Novbe:
    def __init__(self):
        self.bas = None
        self.son = None
        self.say = 0

    def bosdur(self):
        return self.bas is None

    def elave_et(self, sernisin):
        yeni_node = Node(sernisin)
        if self.bas is None:
            self.bas = self.son = yeni_node
        else:
            self.son.next = yeni_node
            self.son = yeni_node
        self.say += 1

    def cixar(self):
        if self.bas is None:
```

Console | I/O | AI Agent

```
əli (30 yaş) normal növbəyə əlavə olundu
Aysel (25 yaş) normal növbəyə əlavə olundu
Nəriman (68 yaş) yaşlılar növbəsinə əlavə olundu
Fəridə (70 yaş) yaşlılar növbəsinə əlavə olundu
Yaşlılar növbəsi:
  1. Nəriman (68 yaş)
  2. Fəridə (70 yaş)
Normal növbə:
  1. əli (30 yaş)
  2. Aysel (25 yaş)
Nəriman (68 yaş) avtobusa mindi
Fəridə (70 yaş) avtobusa mindi
əli (30 yaş) avtobusa mindi
Yaşlılar növbəsi:
  (boşdur)
Normal növbə:
  1. Aysel (25 yaş)
```